

Optimisation

OLIVER WEST

Contents

1	Convex optimisation	2
1.1	Convex functions	2
1.1.1	First-order conditions for convexity	2
1.1.2	Second-order conditions for convexity	3
1.2	Gradient descent	4
1.2.1	Smoothness and strong convexity	5
1.2.2	Convergence of gradient descent	6
1.3	Newton's method	7
1.3.1	Barrier method	8
2	Lagrange multipliers	9
2.1	Weak and strong duality	12
3	Linear programming	15
3.1	Optimality conditions	16
3.2	Standard form for linear programs	16
3.3	Solving linear programs	17
3.4	Basic solutions and basic feasible solutions	17
3.5	Naive solutions	18
3.6	Towards the simplex method	19
3.7	The simplex method	20
4	Applications of linear programs	21
4.1	Game theory	21
4.1.1	Two-person zero-sum games	21
4.1.2	Pure and mixed strategies	22
4.2	Network flows	24
4.2.1	The transport problem	25
4.2.2	The transport algorithm	26
4.3	Max flow/min cut problem	28
4.3.1	Max flow	28
4.3.2	Min cut	28

Lecture 1 – 01/05/26

What is optimisation? We aim to minimize $f(x)$ over $x \in X$ subject to $h(x) = b$, or $h(x) \leq b$, where comparison in $\mathbb{R}^n$ is pointwise. Terminology is as follows:

- $f(x)$ is called the *objective* function;
- $h(x) = b$ or $h(x) \leq b$ is called the *functional* constraint;
- $x \in X$ is called the *regional* constraint;
- $\chi(b) = \{x : h(x) = b, x \in X\}$ is called the *feasible set*;
- x^* which solves the problem is called the *optimal solution*, and $f(x^*)$ is called the *optimal value* or *optimal cost*.

Proposition 0.1 (Inequality constraints and equality constraints)

Any inequality constraint may be converted into an equality constraint with a *slack variable* s –

$$h(x) \leq b \iff h(x) + s = b, s \geq 0$$

Hence, it is sufficient to consider equality constraints.

§1 Convex optimisation

§1.1 Convex functions

Definition 1.1 (Convex sets). A set $S \subseteq \mathbb{R}^n$ is called *convex* if, for every $x, y \in S$ and $\lambda \in [0, 1]$, the point $\lambda x + (1 - \lambda)y \in S$ – that is, every line segment with endpoints in S is contained in S .

Definition 1.2 (Convex functions). A function $f : S \rightarrow \mathbb{R}$ is called *convex* if S is convex and, for every $x, y \in S$ and $\lambda \in [0, 1]$,

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$$

Intuitively, such functions are U-shaped. Also, if $-f$ is convex, then f is called *concave*.

§1.1.1 First-order conditions for convexity

Taking $y - x \rightarrow 0$, the condition for convexity becomes in terms of a tangent, as opposed to a secant, line. This motivates the following theorem.

Theorem 1.3 (First-order condition)

A (differentiable) function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is convex if and only if, for every $x, y \in S$,

$$f(y) \geq f(x) + (y - x)^T \nabla f(x)$$

Remark. It follows from this that, if $\nabla f(x) = 0$, x is the minimum of f .

Proof. $\implies$ When $n = 1$, we have

$$\begin{aligned} f(x + t(y - x)) &\leq (1 - t)f(x) + tf(y) \\ \implies f(y) &\geq f(x) + \frac{f(x + t(y - x)) - f(x)}{t} \end{aligned}$$

Now, take $t \rightarrow 0$ – the quotient becomes $(y - x)$ times the derivative due to the fact that t is scaled by $y - x$.

To resolve the general case, set $g(t) = f((1 - t)x + ty)$, i.e. restrict f to a segment between x and y . Clearly g is convex, and is from $[0, 1] \rightarrow \mathbb{R}$. Using the 1D result, we have

$$\begin{aligned} g(1) &\geq g(0) + (1 - 0)g'(0) \\ \implies f(y) &\geq f(x) + (y - x)^\top \nabla f(x) \end{aligned}$$

$\Leftarrow$ Let $x_t = (1 - t)x + ty$. Assuming the first-order condition, we have

$$\begin{aligned} f(x) &\geq f(x_t) + (x - x_t)^\top \nabla f(x_t) \\ f(y) &\geq f(x_t) + (y - x_t)^\top \nabla f(x_t) \end{aligned}$$

Multiplying the first inequality by $1 - t$, the second by t , and summing, we obtain

$$(1 - t)f(x) + tf(y) \geq f(x_t) + 0 = f((1 - t)x + ty)$$

■

§1.1.2 Second-order conditions for convexity

Definition 1.4 (Hessian). The *Hessian* of $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is $\nabla^2 f(x)$, given by

$$[\nabla^2 f(x)]_{ij} = \frac{\partial^2 f(x)}{\partial x_i \partial x_j}$$

Proposition 1.5 (Multivariate Taylor series)

The multivariate Taylor series of a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is given by

$$f(y) = f(x) + (y - x)^\top \nabla f(x) + \frac{1}{2}(y - x)^\top \nabla^2 f(x)(y - x) + \dots$$

Proof. Apply 1D Taylor's theorem to $g(t) = f(x + t(y - x))$. We obtain

$$\begin{aligned} g(1) &= g(0) + g'(0) + \frac{1}{2}g''(0) + \dots \\ \implies f(y) &= f(x) + \dots \end{aligned}$$

■

Definition 1.6 (Positive semi-definite matrices). We say a symmetric matrix A is *positive semidefinite*, written $A \succeq 0$, if $x^\top A x \geq 0$ for any x . Equivalently, all the eigenvalues of A are non-negative

Theorem 1.7 (Second-order condition)

A twice-differentiable function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is convex if and only if $\nabla^2 f(x) \geq 0$ for all $x \in \mathbb{R}^n$.

Proof. We'll only prove the backward implication. For any x and y , we have

$$\begin{aligned} f(y) &= f(x) + (y - x)^\top \nabla f(x) + \frac{1}{2}(y - x)^\top \nabla^2 f(x)(y - x) \\ &\geq f(x) + (y - x)^\top \nabla f(x) \end{aligned}$$

which is the first-order condition, so f is convex as required. ■

Lecture 2 – 04/05/26**§1.2 Gradient descent**

Recall that, for $y \approx x$,

$$f(y) \approx f(x) + \nabla f(x)^\top (y - x)$$

Let us now suppose $y - x = -\varepsilon \nabla f(x)$, for some $\varepsilon > 0$. Then

$$\begin{aligned} f(y) &\approx f(x) + \nabla f(x)^\top (-\varepsilon \nabla f(x)) \\ &= f(x) - \varepsilon |\nabla f(x)|^2 \leq f(x) \end{aligned}$$

This motivates the following algorithm.

Algorithm 1.8 (Gradient descent) — ① Start from some x_0 and set $t = 0$.

② Repeat:

- Find a *descent direction* v_t (such as $-\nabla f(x_t)$);
- choose a *step size* η_t ;
- update $x_{t+1} = x_t + \eta_t v_t$.

③ Stop when some suitable criteria are satisfied.

What are some potential issues with this algorithm?

- We can ‘overshoot’ our minimum or endlessly oscillate around it if our step size is too large or our function is too sharp;
- or we might never make it to a minimum if our step size is too small or our function is too flat.

We would like to make some new definitions to eliminate these possibilities.

§1.2.1 Smoothness and strong convexity

Definition 1.9 (Matrix ordering). For square matrices of the same size A and B , we say

$$A \leq B \iff B - A \geq 0$$

i.e., $B - A$ is a positive semidefinite matrix. If either A or B is proportional to the identity matrix, this can also be expressed as a bound on the eigenvalues of the other.

Definition 1.10. A twice-differentiable function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is β -smooth if $\nabla^2 f(x) \leq \beta I$ for all $x \in \mathbb{R}^n$. It is α -strongly convex if $\nabla^2 f(x) \geq \alpha I$ for all $x \in \mathbb{R}^n$, $\alpha, \beta > 0$.

Theorem 1.11

If f is β -smooth, then

$$f(y) \leq f(x) + \nabla f(x)^\top (y - x) + \frac{\beta}{2} |y - x|^2$$

and if f is α -s.c., then

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x) + \frac{\alpha}{2} |y - x|^2$$

Proof. Using the Taylor series expansion with remainder, we have

$$f(y) = f(x) + \nabla f(x)^\top (y - x) + \frac{1}{2} (y - x)^\top \nabla^2 f(z) (y - x)$$

where z is some point on the line segment joining x and y . Then, substituting $\nabla^2 f(z)$ for αI or βI with the corresponding inequality gives the result. ■

Corollary 1.12

For a β -smooth function f ,

$$f\left(x - \frac{1}{\beta} \nabla f(x)\right) \leq f(x) - \frac{1}{2\beta} |\nabla f(x)|^2$$

or in other words, $\frac{1}{\beta}$ is a suitably small step size to ensure we do not overshoot the minimum.

Proof. Substitute into Theorem 1.11. ■

Corollary 1.13

Let $f(x^*)$ be the minimum of an α -s.c. f . Then, for any x , we have

$$f(x) - f(x^*) \leq \frac{1}{2\alpha} |\nabla f(x)|^2$$

Remark. It follows that, if $|\nabla f(x)|^2 \leq 2\alpha\varepsilon$, then x is optimal up to an error of ε .

Proof. Take the minimum over y of both sides of Theorem 1.11. We obtain

$$f(x^*) \geq \min_y \left(f(x) + \nabla f(x)^\top (y - x) + \frac{\alpha}{2} |y - x|^2 \right)$$

To find the RHS, let us differentiate. Observe

$$\nabla_y \left(f(x) + \nabla f(x)^\top (y - x) + \frac{\alpha}{2} |y - x|^2 \right) = \nabla f(x) + \alpha(y - x)$$

so setting the gradient to 0 gives

$$y^* = x - \frac{1}{\alpha} \nabla f(x)$$

Plugging this back in, we obtain the result. ■

§1.2.2 Convergence of gradient descent

Theorem 1.14

Let f be α -s.c. and β -smooth, and suppose f is minimised at x^* . Then gradient descent with $\eta_t = \frac{1}{\beta}$ satisfies

$$f(x_T) - f(x^*) = \left(1 - \frac{\alpha}{\beta} \right)^T (f(x_0) - f(x^*)) \leq e^{-\frac{\alpha T}{\beta}} (f(x_0) - f(x^*))$$

since $e^{-x} \geq 1 - x$. Hence, taking

$$T \geq \frac{\beta}{\alpha} \log \left(\frac{f(x_0) - f(x^*)}{\varepsilon} \right)$$

will give an ε -optimal result.

Proof. Have

$$\begin{aligned} f(x_{t+1}) - f(x^*) &\leq f(x_t) - f(x^*) - \frac{1}{2\beta} |\nabla f(x_t)|^2 && \text{by corollary 1.12} \\ &\leq f(x_t) - f(x^*) - \frac{\alpha}{\beta} (f(x_t) - f(x^*)) && \text{by corollary 1.13} \\ &= \left(1 - \frac{\alpha}{\beta} \right) (f(x_t) - f(x^*)) \end{aligned}$$

from which the result clearly follows. ■

Example 1.15

Consider

$$f(x) = \frac{1}{2}(x_1^2 + 100x_2^2)$$

This has

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \leq \nabla^2 f(x) = \begin{pmatrix} 1 & 0 \\ 0 & 100 \end{pmatrix} \leq \begin{pmatrix} 100 & 0 \\ 0 & 100 \end{pmatrix}$$

so $\alpha = 1$, $\beta = 100$. But then $1 - \frac{\alpha}{\beta} = 0.99$, so gradient descent will have bad convergence. But if we just let $x'_2 = 10x_2$, then we immediately have perfect convergence! This is called *preconditioning* our function.

§1.3 Newton's method

Recall the second-order Taylor approximation:

$$f(y) = f(x) + \nabla f(x)^\top(y - x) + \frac{1}{2}(y - x)^\top \nabla^2 f(x)(y - x)$$

Since this is the best possible quadratic approximation, it seems reasonable to try and minimize this quantity. This motivates the update step of

$$x_{t+1} = \underset{y}{\operatorname{argmin}} \left(f(x_t) + \nabla f(x_t)^\top(y - x_t) + \frac{1}{2}(y - x_t)^\top \nabla^2 f(x_t)(y - x_t) \right)$$

To minimize this, let us take the grad with respect to y and make it zero:

$$\begin{aligned} 0 &= \nabla f(x_t) + \nabla^2 f(x_t)(y - x_t) \\ \implies &\boxed{y = x_t - (\nabla^2 f(x_t))^{-1} \nabla f(x_t)} \end{aligned}$$

In 1D, we can make this into a root-finding method for $f'(x)$. Observe

$$\begin{aligned} f'(x) &= g(x) \\ x_{t+1} &= x_t - \frac{g(x_t)}{g'(x_t)} \end{aligned}$$

which looks familiar!

Definition 1.16 (Matrix norm). For $A \in \mathbb{R}^{n \times n}$, the *norm* $|A|$ is the smallest $a > 0$ such that $|Az| \leq a|z|$ for all $z \in \mathbb{R}^n$. For A positive semi-definite, this is just the largest eigenvalue.

Theorem 1.17

If f is a twice-differentiable α -smoothly convex function that has an ℓ -Lipschitz Hessian, i.e. for any $x, y \in \mathbb{R}^n$,

$$|\nabla^2 f(x) - \nabla^2 f(y)| \leq \ell |x - y|$$

then Newton's method satisfies

$$f(x_k) - f(x^*) \leq \frac{2\alpha^3}{\ell^2} \left(\frac{\ell}{2\alpha^2} |\nabla f(x_0)| \right)^{2^{k+1}}$$

Remark. If $|\nabla f(x_0)| < \frac{2\alpha^2}{\ell}$, then this will be getting small extremely extremely fast.

Remark. In applications such as AI, one prefers gradient descent, because calculating the Hessian of functions with 10^9 inputs is just intractable.

§1.3.1 Barrier method

Suppose we want to minimize $f(x)$ subject to a condition $a_i^\top x \leq b_i$ for $1 \leq i \leq m$ (where f is convex). There is a very silly way to do this...

Let

$$g(x) = f(x) + \sum_{i=1}^m \phi(a_i^\top x - b_i)$$

where

$$\phi(x) = \begin{cases} x & x \leq 0 \\ +\infty & \text{o/w} \end{cases}$$

This is cursed, because $\phi(x)$ lacks any reasonable properties. But we can settle for a smoothed-out version of this. For example, let

$$\phi(x) = -\log(-x)$$

called a *logarithmic barrier function*. This is pretty good, but of course minimising g will not now lead exactly to minimizing f . To fix this, introduce a parameter $t > 0$, and consider minimising

$$tf(x) + \sum_{i=1}^m \phi(a_i^\top x - b_i)$$

and gradually increase t . This seems like it should work. Hence, we can write down the following algorithm.

Algorithm 1.18 (Barrier method) — ① Find a strictly feasible x and set $t > 0$, $\alpha > 1$.
② Repeat:

- Compute $x^*(t)$ by minimising

$$tf(x) + \sum_{i=1}^m \phi(a_i^\top x - b_i)$$

using Newton's method with x as the starting point.

- Update $x := x^*(t)$, and set $t := \alpha t$.
- When t is sufficiently large, stop.

§2 Lagrange multipliers

Definition 2.1 (Lagrangian). For an optimisation problem, the *Lagrangian* is given by

$$L(x, \lambda) = f(x) - \lambda^\top (h(x) - b)$$

We now consider the unconstrained problem of minimising $L(x, \lambda)$.

Theorem 2.2 (Lagrange sufficiency)

Let $x^* \in X$ such that $h(x^*) = b$, and $\lambda^* \in \mathbb{R}^m$ such that

$$L(x^*, \lambda^*) = \min_{x \in X} L(x, \lambda^*)$$

Then x^* also solves the original problem.

Proof. Suppose (x^*, λ^*) is as given in the statement. Clearly

$$\min_{x \in X(b)} f(x) \leq f(x^*)$$

since $x^* \in X(b)$. To prove the reverse inequality,

$$\begin{aligned} \min_{x \in X(b)} f(x) &= \min_{x \in X(b)} f(x) - \underbrace{\lambda^{*\top} (h(x) - b)}_0 \\ &\geq \min_{x \in X} \underbrace{f(x) - \lambda^{*\top} (h(x) - b)}_{L(x, \lambda^*)} \\ &= f(x^*) - \lambda^{*\top} (h(x^*) - b) \\ &= f(x^*) \end{aligned}$$

as required. ■

Example

Let us attempt to minimise

$$f(x) = -x_1 - x_2 + x_3$$

subject to

$$\begin{aligned} x_1^2 + x_2^2 &= 4 \\ x_1 + x_2 + x_3 &= 1 \end{aligned}$$

Have

$$\begin{aligned} L(x, \lambda) &= -x_1 - x_2 + x_3 - \lambda_1(x_1^2 + x_2^2 - 4) - \lambda_2(x_1 + x_2 + x_3 - 1) \\ &= [(-1 - \lambda_2)x_1 - \lambda_1 x_1^2] + [(-1 - \lambda_2)x_2 - \lambda_1 x_2^2] + [(1 - \lambda_2)x_3] + 4\lambda_1 + \lambda_2 \end{aligned}$$

Note that we must have $1 - \lambda_2 = 0 \implies \lambda_2 = 1$, otherwise we could take $x_3 \rightarrow \infty$ and L would have no minimum. Hence we have

$$L(x, \lambda) = (-2x_1 - \lambda_1 x_1^2) + (-2x_2 - \lambda_1 x_2^2) + 4\lambda_1 + 1$$

We also need $\lambda_1 \leq 0$ for boundedness and, taking derivatives, we obtain

$$x_1 = x_2 = -\frac{1}{\lambda_1}$$

so, for $\lambda = (\lambda_1, 1)$, we obtain $x^* = (-\frac{1}{\lambda_1}, -\frac{1}{\lambda_1}, x_3)$ for any x_3 . In order to find x_3 , let us apply the feasibility conditions of the original problem – we obtain

$$\begin{aligned} \frac{1}{\lambda_1^2} + \frac{1}{\lambda_1^2} &= 4 \\ -\frac{1}{\lambda_1} - \frac{1}{\lambda_1} + x_3 &= 1 \end{aligned}$$

which gives

$$x^* = (\sqrt{2}, \sqrt{2}, 1 - 2\sqrt{2})$$

Then Lagrange sufficiency allows us to conclude that x^* solves the original problem.

Algorithm 2.3 (Method of Lagrange multipliers) — Without loss of generality, we may assume an equality constraint $h(x) = b$ by adding slack variables $s \geq 0$.

① Set up

$$L(x, s, \lambda) = f(x) - \lambda^\top (h(x) + s - b)$$

② Let

$$\Lambda = \{\lambda \in \mathbb{R}^m : \min L(x, s, \lambda) > -\infty\}$$

- ③ For each $\lambda \in \Lambda$, find $x^*(\lambda)$, $s^*(\lambda)$ which minimise L for the unconstrained problem (still requiring $s \geq 0$).
- ④ Find λ^* such that $(x^*(\lambda^*), s^*(\lambda^*))$ are feasible, and $s^* \geq 0$.

Often this last step can be difficult. Let us introduce the notion of *complementary slackness* to help. Recall the Lagrangian

$$L(x, s, \lambda) = f(x) - \lambda^T(h(x) - b) - \lambda^T s$$

We must have every coordinate of $\lambda \leq 0$, otherwise we could make the corresponding coordinate of s very large and there would be no minimum. Furthermore, it's clear that, if a particular $\lambda_i < 0$, then the Lagrangian is minimised at $s_i = 0$. This is complementary slackness.

Example

Let us attempt to minimise

$$f(x) = x_1 - 3x_2^2$$

subject to

$$\begin{aligned} x_1^2 + x_2^2 &\leq 4 \\ x_1 + x_2 &\leq 2 \end{aligned}$$

Have

$$\begin{aligned} L(x, s, \lambda) &= x_1 - 3x_2 - \lambda_1(x_1^2 + x_2^2 + s_1 - 4) - \lambda_2(x_1 + x_2 + s_2 - 2) \\ &= [(1 - \lambda_2)x_1 - \lambda_1 x_1^2] + [(-3 - \lambda_2)x_2 - \lambda_1 x_2^2 - \lambda_1 s_1 - \lambda_2 s_2 + 4\lambda_1 + 2\lambda_2] \end{aligned}$$

It is now necessary to study all four possible combinations of the $\lambda_i = 0$ and the $\lambda_i < 0$. Let us examine just the first case, with $\lambda_1, \lambda_2 < 0$, so that $s_1 = s_2 = 0$ by complementary slackness. Differentiating, we obtain

$$\begin{aligned} 1 - 2\lambda_1 x_1 - \lambda_2 &= 0 \\ -3 - 2\lambda_1 x_2 - \lambda_2 &= 0 \\ x_1^2 + x_2^2 &= 4 \\ x_1 + x_2 &= 2 \end{aligned}$$

The quadratic constraint gives $(x_1, x_2) = (0, 2)$ or $(x_1, x_2) = (2, 0)$. But we still need to solve for λ_1 and λ_2 – we obtain in each case that one of the λ_i is positive, contradicting our assumption. So there are no solutions here.

Lecture 5? – 11/05/25

Let us look at a general method to solve the Lagrangian, and let us start by counting how many equations and unknowns we have. We have

$$\begin{cases} \frac{\partial L(x, s, \lambda)}{\partial x_i} = 0 & 1 \leq i \leq n \\ h(x)_j + s_j = b_j & 1 \leq j \leq m \\ \lambda_j s_j = 0 & 1 \leq j \leq m \end{cases}$$

i.e. $2n + m$ equations. And, indeed, we have

$$\underbrace{n}_x + \underbrace{m}_\lambda + \underbrace{m}_s = m + 2n$$

variables as required.

Then, if necessary, we can always split into every possible combination of conditions chosen from

$$\begin{array}{ll} \lambda_1 = 0 & \lambda_1 < 0 \\ \vdots & \\ \lambda_m = 0 & \lambda_m < 0 \end{array}$$

Remark. The cases outlined above are called the *Karush-Kuhn-Tucker (KKT) conditions*.

§2.1 Weak and strong duality

Definition 2.4 (Dual objective function). Given an optimisation problem, the *dual objective function* is defined as

$$g(\lambda) = \min_{x \in X} L(x, \lambda)$$

where

$$\Lambda = \{\lambda : \min_{x \in X} L(x, \lambda) > -\infty\}$$

Definition 2.5 (Dual problem). The *dual problem* is to maximise $g(\lambda)$ subject to $\lambda \in \Lambda$

Remark. It is much easier to solve the dual problem. For every fixed x , the Lagrangian linear in λ !

$$L(x, \lambda) = f(x) - \lambda^\top (h(x) - b)$$

Indeed, this is just concave!

Theorem 2.6 (Weak duality)

If $x \in X(b)$ and $\lambda \in \Lambda$, then $f(x) \geq g(\lambda)$. In particular,

$$\min_{x \in X(b)} f(x) \geq \max_{\lambda \in \Lambda} g(\lambda)$$

Remark. If $f(x) = g(\lambda)$ for some (x, λ) , it follows that x minimises f .

Proof. For any $\lambda \in \Lambda$ and $x \in X(b)$,

$$f(x) \geq \min_{x \in X(b)} f(x)$$

$$\begin{aligned}
 &= \min_{x \in X(b)} f(x) - \lambda^\top(h(x) - b) \\
 &\geq \min_{x \in X} f(x) - \lambda^\top(h(x) - b) \\
 &= \min_{x \in X} L(x, \lambda) \\
 &= g(\lambda)
 \end{aligned}$$

Now we can just take the min over x and max over λ , since this holds for every pair (x, λ) . ■

Definition 2.7 (Duality gap). The *duality gap* is the difference between the two quantities above:

$$\min_{x \in X(b)} f(x) - \max_{\lambda \in \Lambda} g(\lambda)$$

If the duality gap is zero, then we say *strong duality* holds.

Theorem 2.8 (Strong duality)

The Lagrange method works if and only if strong duality holds.

Proof. $\implies$ If the Lagrange method works, we have a pair (x^*, λ^*) such that

$$\operatorname{argmin}_{x \in X} L(x, \lambda^*) = x^* \quad h(x^*) = b$$

Then

$$\begin{aligned}
 g(\lambda^*) &= \min_{x \in X} f(x) - (\lambda^*)^\top(h(x) - b) \\
 &= f(x^*) - (\lambda^*)^\top(h(x^*) - b) \\
 &= f(x^*)
 \end{aligned}$$

Hence, since we have found a pair (x^*, λ^*) with $f(x^*) = g(\lambda^*)$, then the duality gap is zero as required.

$\Leftarrow$ Suppose we have $x \in X(b)$, $\lambda \in \Lambda$ such that $f(x) = g(\lambda)$. Then the Lagrange method works using this λ . ■

Definition 2.9. A function $\phi : \mathbb{R}^m \rightarrow \mathbb{R}$ is said to have a *supporting hyperplane* at a point $b \in \mathbb{R}^m$ if there exists λ such that, for all $c \in \mathbb{R}^m$,

$$\phi(c) \geq \phi(b) + \lambda^\top(c - b)$$

Definition 2.10. Let $\phi : \mathbb{R}^m \rightarrow \mathbb{R}$ be defined as follows:

$$\phi(c) = \min_{x \in X(c)} f(x)$$

where $X(c) = \{x : h(x) = c, x \in X\}$.

Theorem 2.11

A problem satisfies strong duality if the value function ϕ has a supporting hyperplane at b . Conversely, if strong duality holds for some λ ($g(\lambda) = \phi(b)$), then λ defines a supporting hyperplane to ϕ at b .

Remark. It is a consequence of this that, if ϕ is convex, the Lagrange method will always work.

Proof. $\Leftarrow$ Suppose ϕ has a supporting hyperplane at b given by λ , so

$$\phi(c) \geq \phi(b) + \lambda^\top(c - b)$$

Hence,

$$\begin{aligned} g(\lambda) &= \min_{x \in X} f(x) - \lambda^\top(h(x) - b) \\ &= \min_c \min_{x \in X(c)} f(x) - \lambda^\top(h(x) - c) - \lambda^\top(c - b) \end{aligned}$$

Now observe that $h(x) - c = 0$ for $x \in X(c)$, and, by definition, $\phi(c) = \min_{x \in X(c)} f(x)$. Hence this is

$$\begin{aligned} g(\lambda) &= \min_c \phi(c) - \lambda^\top(c - b) \\ &\geq \phi(b) \end{aligned}$$

as required. Now, by weak duality, $g(\lambda) \leq \phi(b)$. Hence $g(\lambda) = \phi(b)$, strong duality holds.

$\Rightarrow$ Suppose $g(\lambda) = \phi(b)$ for some λ . Then

$$\begin{aligned} \phi(b) &= g(\lambda) = \min_{x \in X} f(x) - \lambda^\top(h(x) - b) \\ &= \min_{x \in X} f(x) - \lambda^\top(h(x) - c) - \lambda^\top(c - b) \\ &\leq \min_{x \in X(c)} f(x) - \lambda^\top(h(x) - c) - \lambda^\top(c - b) \\ &= \phi(c) - \lambda^\top(c - b) \end{aligned}$$

as required. ■

Theorem 2.12

The following problem has a convex value function ϕ :

$$\begin{aligned} \min & f(x) \\ \text{s.t. } & h_j(x) \leq b_j \quad 1 \leq j \leq m \\ & x \in X \end{aligned}$$

where X is convex, f is convex, and each h_j is convex.

Proof. See example sheet. ■

§3 Linear programming

Definition 3.1 (Linear programs). A *linear program* is a problem of the following form:

$$\begin{array}{ll}
 \min c^\top x & \\
 a_i^\top x \geq b_i & i \in M_1 \\
 a_i^\top x \leq b_i & i \in M_2 \\
 a_i^\top x = b_i & i \in M_3 \\
 x_j \geq 0 & j \in N_1 \\
 x_j \leq 0 & j \in N_2 \\
 x_j = 0 & j \in N_3
 \end{array}$$

Note $m = M_1 + M_2 + M_3$ is the number of constraints, and $n = N_1 + N_2 + N_3$ is the dimension of the feasible region.

It is sometimes helpful to put the a_i into a matrix, as follows:

$$\begin{pmatrix} - & a_1^\top & - \\ & \vdots & \\ - & a_m^\top & - \end{pmatrix}$$

Theorem 3.2 (Duals of linear programs)

The dual of the linear program above is given by

$$\min b^\top x$$

in a set Λ as given below.

Proof. Adding slack variables with the relevant signs, we obtain

$$\begin{aligned}
 L(x, s, \lambda) &= c^\top x - \sum_{i \in M_1} \lambda_i (a_i^\top x - s_i - b_i) - \sum_{i \in M_2} \lambda_i (a_i^\top x + s_i - b_i) - \sum_{i \in M_3} \lambda_i (a_i^\top x - b_i) \\
 &= \sum_{j \in N_1} (c_j - \lambda^\top A_j) x_j + \sum_{j \in N_2} (c_j - \lambda^\top A_j) x_j + \sum_{j \in N_3} (c_j - \lambda^\top A_j) x_j \\
 &\quad + \sum_{i \in M_1} \lambda_i s_i - \sum_{i \in M_2} \lambda_i s_i + \sum_{i \in M_1 \cup M_2 \cup M_3} \lambda_i b_i
 \end{aligned}$$

Then, once again considering what is necessary for us not to be able to send any of the terms to infinity, we obtain the following set of constraints on λ :

$$\begin{array}{ll}
 \lambda_i \geq 0 & i \in M_1 \\
 \lambda_i \leq 0 & i \in M_2 \\
 \lambda_i \text{ free} & i \in M_3 \\
 \lambda^\top A_j \leq c_j & j \in N_1 \\
 \lambda^\top A_j \geq c_j & j \in N_2 \\
 \lambda^\top A_j = c_j & j \in N_3
 \end{array}$$

This is our set Λ . It is clear that, given these constraints, we can make every term 0, which is the minimum possible under these constraints, and we are simply left with

$$g(\lambda) = \sum_{i \in M_1 \cup M_2 \cup M_3} \lambda_i b_i = b^\top \lambda$$

So the dual problem is to maximise $b^\top \lambda$ for $\lambda \in \Lambda$. ■

§3.1 Optimality conditions

Theorem 3.3

Let x and λ be feasible solutions to the primal problem and its duals as above. Then these are optimal if and only if

$$\begin{aligned} \lambda_i(a_i^\top x - b_i) &= 0 \\ (c_j - \lambda^\top A_j)x_j &= 0 \end{aligned}$$

for all i, j .

Remark. This is probably best thought of as analogous to the gradient being zero if and only if we are at the minimum of a convex function.

Proof. $\implies$ Define $u_i = \lambda_i(a_i^\top x - b_i)$, $v_j = (c_j - \lambda^\top A_j)x_j$. Since x and λ are each feasible, we can see $u_i \geq 0$ for all i and $v_j \geq 0$ for all j , by comparing the sign restrictions in each feasible set.

Summing, we obtain

$$\begin{aligned} \sum_i u_i + \sum_j v_j &= (\lambda^\top Ax - \lambda^\top b) + (c^\top x - \lambda^\top Ax) \\ &= c^\top x - \lambda^\top b \end{aligned}$$

By weak duality, we know $c^\top x \geq \lambda^\top b$. If $c^\top x = \lambda^\top b$, i.e. x and λ are optimal, then the LHS is zero, so, since all the quantities on the LHS are non-negative, they are all zero.

$\Leftarrow$ Conversely, if $u_i = 0$ for all i , $v_j = 0$ for all j , then $c^\top x = \lambda^\top b$, so, by weak duality, x and λ are optimal. ■

§3.2 Standard form for linear programs

Observe that we can always write any linear program in *general form*, namely

$$\begin{aligned} \min f(x) \\ Ax \leq b \end{aligned}$$

by fiddling with the signs of the constraints and adding slack variables as relevant. However, it is even better to look at *standard form*, namely

$$\begin{aligned} \min f(x) \\ Ax = b \\ x \geq 0 \end{aligned}$$

Proposition 3.4

Every linear program may be converted into standard form.

Proof. Any real x can be written as

$$x = x_+ - x_-$$

where x_+ and x_- are both positive reals. Then, we can do the conversion as follows:

$$\min c^\top x$$

$$Ax \leq b$$

$$\min c^\top x$$

$$Ax + s = b$$

$$s \geq 0$$

$$\begin{aligned} \min c^\top (x_+ - x_-) \\ A(x_+ - x_-) + s = b \\ x_+, x_-, s \geq 0 \end{aligned}$$

and the final problem is in the desired form. ■

§3.3 Solving linear programs

Minimising $c^\top x$ is the same as maximising $-c^\top x$. But $-c^\top x$ is convex – we’re trying to *maximise* a convex function.

Definition 3.5 (Extreme points). A point $x \in C$, where $C \subset \mathbb{R}^n$ is convex, is called an *extreme point* if it cannot be written as a convex combination of two distinct elements of C .

Observe that the max of a convex function f on a convex set C occurs at an extreme point of C . Indeed, we have

$$f(ty + (1-t)z) \leq tf(y) + (1-t)f(z) \leq \max(f(y), f(z))$$

In general, sets can of course have infinitely many extreme points. But, happily, for a linear program, the constraint set is always a polytope, and hence has finitely many extreme points.

§3.4 Basic solutions and basic feasible solutions

Recall that A is an n by m matrix. Denote its rows $a_1, \dots, a_m$ and its columns $A_1, \dots, A_n$. We make three assumptions.

- ① *The rows of A are linearly independent.* This is without loss of generality, since otherwise the constraints are redundant.
- ② *Every set of m columns of A is linearly independent.* This assumption loses generality, but we can often make it work by adding some random noise to each entry.

Definition 3.6 (Basic solutions and basic feasible solutions). A point x that satisfies $Ax = b$ and has at most m non-zero entries is called a *basic solution*. A basic solution x that satisfies $x \geq 0$ is called a *basic feasible solution*.

Choose indices $\{B(1), \dots, B(m)\}$ such that $x_i = 0$ for all $i \notin \{B(1), \dots, B(m)\}$. Then

$$Ax = \sum_{i=1}^n A_i x_i = \sum_{i=1}^m A_{B(i)} x_{B(i)} = Bx_B$$

where

$$\begin{pmatrix} | & & | \\ A_{B(1)} & \cdots & A_{B(m)} \\ | & & | \end{pmatrix}$$

and

$$x_B = \begin{pmatrix} x_{B(1)} \\ \vdots \\ x_{B(m)} \end{pmatrix}$$

Then

$$Ax = b \iff Bx_B = b \iff x_B = B^{-1}b$$

Terminology is that

- $A_{B(1)}, \dots, A_{B(m)}$ are the *basic columns*;
- $x_{B(1)}, \dots, x_{B(m)}$ are the *basic variables*;
- B is the *basis matrix*.

Note also that, for every choice of B , we obtain a basic solution. If $B^{-1}b \geq 0$, then that basis B leads to a basic feasible solution. Now, we may state the final assumption:

- ③ *Every basic feasible solution is non-degenerate – it has exactly m non-zero entries.*
This isn't essential, and loses generality, but will make our lives easier.

§3.5 Naive solutions

Theorem 3.7

A vector x is a basic feasible solution if and only if it is an extreme point of the set $X(b) = \{x : Ax = b, x \geq 0\}$.

Remark. Now we have a (bad) method to solve linear programs! Find all $\binom{n}{m}$ basic solutions, filter out the basic feasible solutions, and evaluate $c^T x$ at each of these. This is bad, however, since $\binom{n}{m}$ is typically exponential in n .

Proof. $\implies$ Suppose x is a basic feasible solution that can be written as $x = (1-t)y + tz$ for $y \neq z \in X(b)$ and $t \in (0, 1)$.

If $x_i = 0$, then, since

$$x_i = (1-t)y_i + tz_i$$

and y_i and z_i are non-negative, it must be that $y_i = z_i = 0$. Hence y and z are basic solutions corresponding to the same basis as x , so $x = y = z$, contradiction.

$\Leftarrow$ We take the contrapositive. Suppose x is not a basic feasible solution. If $x \notin X(b)$, then clearly it is not an extreme point of $X(b)$, so we assume $x \in X(b)$.

Because x is not a basic feasible solution, it must have $r > m$ positive entries. Suppose $x_{i_1}, \dots, x_{i_r} > 0$, the columns $A_{i_1}, \dots, A_{i_r}$ are linearly dependent, so we have $w_{i_1}, \dots, w_{i_r}$ not all zero with

$$w_{i_1}A_{i_1} + \dots + w_{i_r}A_{i_r} = 0$$

Make a vector w which is zero everywhere except at locations i_j , where $w(i_j) = w_{i_j}$. Then $Aw = 0$. Now consider two new points

$$y = x + \varepsilon w$$

$$z = x - \varepsilon w$$

for a small $\varepsilon > 0$. These certainly have $Ay = Az = b$ and, for sufficiently small ε , we can also ensure that all the entries are non-negative, since we are only changing the non-negative entries of x , and by a sufficiently small amount not to make them negative. But then

$$x = \frac{1}{2}y + \frac{1}{2}z$$

so x is not an extreme point. ■

§3.6 Towards the simplex method

Recall that, for a linear program in standard form, the optimality conditions are

- primal feasibility, $Ax = b$, $x \geq 0$;
- dual feasibility, $A^T\lambda \leq c$;
- complementary slackness, $x^T(c - A^T\lambda) = 0$.

For any basic feasible solution, the complementary slackness equation reduces to

$$x_B^T(c_B - B^T\lambda) = 0$$

But then, since $x_B > 0$ by non-degeneracy, we must have

$$\begin{aligned} c_B &= B^T\lambda \\ \lambda &= (B^T)^{-1}c_B \end{aligned}$$

Then, if

$$\boxed{A^T(B^T)^{-1}c_B \leq c}$$

then λ is dual feasible and hence optimal.

Definition 3.8 (Reduced cost vector). The *reduced cost vector* $\bar{c}$ is defined as

$$\bar{c}^\top = c^\top - c_B^\top B^{-1}A \iff \bar{c}_j = c_j - c_B^\top B^{-1}A_j$$

If, for a basic feasible solution x , $\bar{c} \geq 0$, then x is optimal.

§3.7 The simplex method

Let

$$x = (x_{B(1)}, x_{B(2)}, \dots, x_{B(m)}, 0, \dots, 0)$$

Suppose, for some $j \notin \{B(1), \dots, B(m)\}$, $\bar{c}_j < 0$. Let us attempt to make $c_j > 0$ while keeping all non-basic x_i 's = 0. In other words, we want to perturb

$$x' = x + t(d_{B(1)}, \dots, d_{B(m)}, 0, \dots, 0, 1, 0, \dots, 0)$$

where 1 is in the j^{th} position. But we want $Ax' = b$ so, since $Ax = b$ already, we need to solve

$$A \begin{pmatrix} d_B \\ 0 \\ \vdots \\ 0 \\ \vdots \end{pmatrix} = 0 \iff Bd_B + A_j = 0 \implies d_B = -B^{-1}A_j$$

What, then, is the new cost? We have

$$\begin{aligned} c^\top x' &= c^\top x + t(c_B^\top d_B + c_j) \\ &= c^\top x + t(c_j - c_B^\top B^{-1}A_j) \\ &= c^\top x + t\bar{c}_j \end{aligned}$$

So what should we make t ? Well, since the cost decreases the further we go, we want to make t as large as possible. But, if we go too far, we will leave the feasible set. Hence, we calculate

$$-\frac{x_{B(1)}}{d_{B(1)}}, \dots, -\frac{x_{B(m)}}{d_{B(m)}}$$

and let t^* be the smallest positive of these. Then, for some i ,

$$\begin{aligned} t^* &\leq -\frac{x_{B(i)}}{d_{B(i)}} \\ \implies x_{B(i)} + t^*d_{B(i)} &\geq 0 \end{aligned}$$

so the new solution is still feasible. It remains to check that x' has at most m non-zero entries. Observe that, since $t^* = -\frac{x_{B(i)}}{d_{B(i)}}$ for some i , the i^{th} entry in x' goes to zero, and x'_j has become non-zero. Hence we still have at most m non-zero entries, as required.

Remark. Under the assumption of non-degeneracy mentioned earlier, we actually know that the i is unique, so x' has exactly m non-zero entries.

In practice, the algorithm goes as follows:

Algorithm 3.9 (Simplex algorithm) — ① Sitting at a basic feasible solution x , we construct the *simplex tableau* as follows:

$-c_B^T x_B$	$\bar{c}_1$	$\cdots$	$\bar{c}_n$
$x_{B(1)}$		$\cdots$	
$\vdots$	$B^{-1}A_1$	$\cdots$	$B^{-1}A_n$
$x_{B(1)}$		$\cdots$	

② We find

- the *pivot column* – any with $\bar{c}_i < 0$;
- the *pivot row* – with the minimal value of $\frac{x_{B(j)}}{(B^{-1}A_j)(j)}$;
- the *pivot entry*, at the intersection of these.

and then perform row operations to make the pivot column all zeroes except in the pivot entry.

③ Continue until $\bar{c} \geq 0$.

Remark (History of the simplex algorithm). The simplex algorithm was developed by George Danzig while working for the US military in 1947. It was practical to run even on computers at that time, and so it was assumed that it ran in polynomial time.

However, Klee and Minty in 1970 found that certain pathological cases could take exponential time. Indeed, in 1979, Khachiyan's ellipsoid algorithm showed that linear programs are in P – but the algorithm was useless in practice.

Then, in 1984, Narendra Karmarkar invented his eponymous algorithm, which turned out to be a special case of barrier methods innovated by Soviet mathematicians in the 60s.

§4 Applications of linear programs

§4.1 Game theory

§4.1.1 Two-person zero-sum games

Definition 4.1 (Payoff matrix). For a two-person zero-sum game, the *payoff matrix* is an m by n matrix A such that, if player 1 plays action i and player 2 plays action j , then player 1 gains A_{ij} points and player 2 loses A_{ij} points.

Example (Rock-paper-scissors)

For rock-paper-scissors, the payoff matrix is

$$\begin{pmatrix} 0 & -1 & 1 \\ 1 & 0 & -1 \\ -1 & 1 & 0 \end{pmatrix}$$

where actions 1, 2, and 3 are rock, paper, scissors respectively.

If P1 plays i , then they can expect to have a payoff of

$$\min_{j \in \{1, \dots, n\}} A_{ij}$$

Hence, they should try and solve

$$\max_{i \in \{1, \dots, m\}} \min_{j \in \{1, \dots, n\}} A_{ij} \quad (\dagger)$$

Conversely, P2 should solve

$$\min_{j \in \{1, \dots, n\}} \max_{i \in \{1, \dots, m\}} A_{ij} \quad (\ddagger)$$

Example

Suppose

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Then, P1 should play $i = 2$, and P2 should play $j = 1$, so 3 will be chosen every time. In other words, $(\dagger)$ and $(\ddagger)$ have the same solution. This is called a *saddle point*.

Example

Suppose instead

$$A = \begin{pmatrix} 4 & 2 \\ 1 & 3 \end{pmatrix}$$

Then P1 should play $i = 1$ in which case P2 would play $j = 2$ – but, P2 should play $j = 2$ in which case P1 would play $i = 2$. Indeed, the solutions to $(\dagger)$ and $(\ddagger)$ are not the same.

§4.1.2 Pure and mixed strategies

A *pure strategy*, which we have considered up to this point, is when the player in question, say P1, picks a fixed action. Alternatively, P1 could pick a distribution $p = (p(1), \dots, p(m))^T$ and play their actions according to these probabilities. But we can do a similar analysis! P1's expected payoff is

$$\min_{j \in \{1, \dots, n\}} \sum_i A_{ij} p_i$$

so P1 looks to solve

$$\max_p \min_{j \in \{1, \dots, n\}} \sum_i A_{ij} p_i$$

Indeed, we can re-express this as

$$\max v$$

$$\begin{aligned} A^\top p &\geq ve \\ e^\top p &= 1 \\ p &\geq 0 \end{aligned}$$

where e is the vector of all ones. Likewise, P2's problem is

$$\begin{aligned} \min w \\ Aq &\leq we \\ e^\top q &= 1 \\ q &\geq 0 \end{aligned}$$

Brilliantly, it turns out that these two problems are dual. Importantly, this means there will always be a saddle point. Observe

$$\begin{aligned} L(w, q, s, \lambda_1, \lambda_2) &= w + \lambda_1^\top (Aq + s - we) - \lambda_2^\top (e^\top q - 1) \\ &= w(1 - \lambda_1^\top e) + (\lambda_1^\top A - \lambda_2 e^\top)q + \lambda_1^\top s + \lambda_2 \end{aligned}$$

Hence, Λ has

- $\lambda_1^\top e = 1$;
- $\lambda_1^\top A \geq \lambda_2 e^\top$;
- $\lambda_1 \geq 0$.

Then, when $\lambda \in \Lambda$, $\min L = \lambda_2$. The dual, then, is

$$\begin{aligned} \max \lambda_2 \\ A^\top \lambda_1 &\geq \lambda_2 e \\ \lambda_1^\top e &= 1 \\ \lambda_1 &\geq 0 \end{aligned}$$

– familiar!

Theorem 4.2 (Optimal strategies in two-player zero-sum games)

A mixed strategy p is optimal for P1 if and only if there exists a strategy q for P2 and a number v such that

- $A^\top p \geq ve$, $e^\top p = 1$, $p \geq 0$ (primal feasibility);
- $Aq \leq ve$, $e^\top q = 1$, $q \geq 0$ (dual feasibility);
- $v = p^\top Aq$ (complementary slackness).

Proof. We have already established the duality of the problems. It remains only to check complementary slackness. If (p, v) and (q, w) are primal resp. dual optimal, then we need

$$\begin{aligned} (Aq - we)^\top p &= 0 \\ q^\top (A^\top p - ve) &= 0 \end{aligned}$$

giving

$$p^\top Aq = w = v$$

as required. ■

Notably, the difference between playing first and second that we had with fixed strategies disappears in the case of fixed strategies.

In practice, how can we apply these techniques to solve a game?

- ① Look for a saddle point.
- ② Look for dominating actions – whose payoffs are always greater than another for the same response. We can drop rows and columns which are dominated.
- ③ Solve the LP, either graphically or using the simplex method, to find the best mixed strategy.

Example

Suppose the payoff matrix is

$$\begin{pmatrix} 2 & 3 & 4 \\ 3 & 1 & 1/2 \\ -1 & 3 & 2 \end{pmatrix}$$

Observe that every entry in row 1 is greater than the corresponding entry in row 3, so row 3 is dominated and we can look at the submatrix

$$\begin{pmatrix} 2 & 3 & 4 \\ 3 & 1 & 1/2 \end{pmatrix}$$

Hence, P1's mixed strategy is $(p, 1 - p, 0)$. The problem, then, is

$$\begin{aligned} \max v \\ \begin{cases} 2p + 3(1 - p) \geq v \\ 3p + (1 - p) \geq v \\ 4p + \frac{1}{2}(1 - p) \geq v \\ 0 \leq p \leq 1 \end{cases} \end{aligned}$$

We can solve this graphically to obtain the optimal solution $(2/3, 1/3, 0)$, which satisfies optimality conditions.

§4.2 Network flows

Definition 4.3 (Directed graph). A *directed graph* is $G = (V, E)$, where V is the set of vertices and $E \subseteq V \times V$ is the set of edges.

In the *minimum cost flow problem*, we define a *flow* x_{ij} for each edge $(i, j) \in E$. We also define

- a vector $b \in \mathbb{R}^n$ such that b_i is the net flow leaving vertex i , where flow into i is counted as negative;
- a cost matrix C where C_{ij} is the cost per unit flow along edge (i, j) ;
- optionally, matrices $\bar{M}$ and $\underline{M}$ which give the maximal and minimal flow along each edge, respectively.

The problem, then, is

$$\begin{aligned} \min \quad & \sum c_{ij}x_{ij} \\ b_i + \sum_{(j,i) \in E} x_{ji} = \sum_{(i,j) \in E} x_{ij} \\ \underline{M}_{ij} \leq x_{ij} \leq \bar{M}_{ij} \end{aligned}$$

§4.2.1 The transport problem

Here, we have n suppliers $i \in \{1, 2, \dots, n\}$ and m consumers $j \in \{1, 2, \dots, m\}$ – each supplier i produces s_i goods, and consumer j demands d_j goods, where $\sum s_i = \sum d_j$. Also, the graph is complete bipartite between the suppliers and consumers. The problem becomes

$$\begin{aligned} \min \quad & \sum c_{ij}x_{ij} \\ \sum_j x_{ij} &= s_i & 1 \leq i \leq n \\ \sum_i x_{ij} &= d_j & 1 \leq j \leq m \\ x_{ij} &\geq 0 \end{aligned}$$

Excitingly, this is already in standard form, although some of the constraints are redundant. Indeed, what is really exciting is the next theorem.

Theorem 4.4

Every minimum cost flow problem with finite capacities can be recast as an equivalent transport problem.

Remark. If $C_{ij} \geq 0$ for every (i, j) , then we can assume $M_{ij} < \infty$ without loss of generality. Indeed, we can just make $M_{ij} = \sum_{i \in V} |b_i|$.

Proof. We can set

$$\begin{aligned} \tilde{x}_{ij} &= x_{ij} - \underline{M}_{ij} \\ \implies 0 \leq \tilde{x}_{ij} &\leq \bar{M}_{ij} - \underline{M}_{ij} \end{aligned}$$

Now $\sum c_{ij}x_{ij} = \sum c_{ij}\tilde{x}_{ij} + \sum \underline{M}_{ij}c_{ij}$. Then, we can rewrite the flow condition by putting

$$\tilde{b}_i = b_i + \sum_{(j,i) \in E} \underline{M}_{ij} - \sum_{(i,j) \in E} \underline{M}_{ij}$$

Hence, we assume wlog $\underline{M}_{ij} = 0$. Now, for every vertex i , construct a consumer, and for every edge (i, j) , construct a supplier. Define

$$\begin{aligned} s_{ij} &= \bar{M}_{ij} \\ d_i &= \sum_{k: (i,k) \in E} \bar{M}_{ik} - b_i \end{aligned}$$

$$\begin{aligned} C_{(i,j) \rightarrow i} &= 0 \\ C_{(i,j) \rightarrow j} &= c_{ij} \end{aligned}$$

Letting x_{ij} be the flow from vertex (i, j) to vertex j , we can see $\bar{M}_{ij} - x_{ij}$ is the flow from vertex (i, j) to vertex i . Observe that the total flow into consumer i is

$$\sum_{k:(i,k) \in E} (\bar{M}_{ik} - x_{ik}) + \sum_{k:(k,i) \in E} x_{ki}$$

and the total demand is

$$\sum_{k:(i,k) \in E} \bar{M}_{ik} - b_i$$

Setting these equal, we obtain

$$\sum_{k:(i,k) \in E} x_{ik} = b_i + \sum_{k:(k,i) \in E} x_{ki}$$

which is exactly the flow condition from the original problem. Furthermore, for any solution to the transport problem, the cost is $\sum c_{ij}x_{ij}$ as required. ■

§4.2.2 The transport algorithm

Recall the statement of the transport problem. Let us apply the optimality conditions to obtain

Theorem 4.5 (Optimality for the transport problem)

If, for some feasible x , we have dual variables $\lambda \in \mathbb{R}^n$ and $\mu \in \mathbb{R}^m$ such that

- $c_{ij} \geq \lambda_i + \mu_j$;
- $(c_{ij} - (\lambda_i + \mu_j)) x_{ij} = 0$

for all i, j , then x is optimal.

Proof. The Lagrangian is

$$\begin{aligned} L(x, \lambda, \mu) &= \sum_i \sum_j c_{ij} x_{ij} - \sum_i \lambda_i \left(\sum_j x_{ij} - s_i \right) - \sum_j \mu_j \left(\sum_i x_{ij} - d_j \right) \\ &= \sum_i \sum_j (c_{ij} - \lambda_i - \mu_j) x_{ij} + \sum_i \lambda_i s_i + \sum_j \mu_j d_j \end{aligned}$$

Then, we can read off the conditions as dual feasibility and complementary slackness, respectively. The result follows. ■

Remark. For a given basic feasible solution x , when we solve for (λ, μ) using complementary slackness, we may without loss of generality take $\lambda_1 = 0$.

We thus have the following algorithm:

Example

Take

$$(s_1, s_2, s_3) = (14, 10, 9)$$

$$(d_1, d_2, d_3, d_4) = (12, 5, 8, 8)$$

We can easily construct a basic feasible solution, for example by alternately satisfying demand and supply of increasing consumers and suppliers.

Take also

$$C = \begin{pmatrix} 5 & 3 & 4 & 6 \\ 2 & 7 & 4 & 1 \\ 5 & 6 & 2 & 4 \end{pmatrix}$$

giving the tableau

	μ_1				μ_2				μ_3				μ_4				
λ_1	$\lambda_1 + \mu_1$	12	5		$\lambda_1 + \mu_2$	2	3		$\lambda_1 + \mu_3$	0	4		$\lambda_1 + \mu_4$	0	6		14
λ_2	$\lambda_2 + \mu_1$	0	2		$\lambda_2 + \mu_2$	3	7		$\lambda_2 + \mu_3$	7	4		$\lambda_2 + \mu_4$	0	1		10
λ_3	$\lambda_3 + \mu_1$	0	5		$\lambda_3 + \mu_2$	0	6		$\lambda_3 + \mu_3$	1	2		$\lambda_3 + \mu_4$	8	4		9
		12				5				8				8			

where the λ_i and μ_j are as yet unknown. We obtain

$$\begin{cases} \lambda_1 + \mu_1 = 5 \\ \lambda_2 + \mu_2 = 7 \\ \lambda_3 + \mu_3 = 2 \\ \lambda_1 + \mu_2 = 3 \\ \lambda_2 + \mu_3 = 4 \\ \lambda_3 + \mu_4 = 4 \end{cases}$$

Recalling we may without loss of generality take $\lambda_1 = 0$, we obtain

$$\begin{aligned} \lambda_1 &= 0 & \mu_1 &= 5 \\ \lambda_2 &= 4 & \mu_2 &= 3 \\ \lambda_3 &= 2 & \mu_3 &= 0 \\ & & \mu_4 &= 2 \end{aligned}$$

Hence we have the following tableau:

	5				3				0				2				
0	5	12	5		3	2	3		0	0	4		2	0	6		14
4	9	0	2		7	3	7		4	7	4		6	0	1		10
2	7	0	5		5	0	6		2	1	2		4	8	4		9
		12				5				8				8			

Now, we find one edge which we want to add some mass to (by checking where $\lambda_i + \mu_j > c_{ij}$), and adjust the other edges to keep the total flow the same, by finding a cycle containing that edge. In particular, we add as much as possible without making any transports negative.

§4.3 Max flow/min cut problem

§4.3.1 Max flow

Given $G = (V, E)$ with source vertex 1 and sink vertex n ($|V| = n$), the max flow problem is

$$\begin{aligned} \max_{x, \delta} \quad & \delta \\ \sum_{\{j: (1,j) \in E\}} \quad & x_{1j} = \delta \\ \sum_{\{i: (i,n) \in E\}} \quad & x_{in} = \delta \\ \sum_{\{j: (i,j) \in E\}} \quad & x_{ij} = \sum_{\{j: (j,i) \in E\}} x_{ji} \\ & 0 \leq x_{ij} \leq c_{ij} \end{aligned}$$

Alternatively, we can define

$$\phi_i(x) = \sum_{\{j: (i,j) \in E\}} x_{ij} - \sum_{\{j: (j,i) \in E\}} x_{ji}$$

whereupon the problem becomes

$$\begin{aligned} \max \quad & \delta \\ \phi_1(x) \quad & = \delta \\ \phi_n(x) \quad & = -\delta \\ \phi_i(x) \quad & = 0 \\ & 0 \leq x_{ij} \leq c_{ij} \end{aligned}$$

and indeed we can also make

$$b_i = \begin{cases} +1 & i = 1 \\ -1 & i = n \\ 0 & \text{o/w} \end{cases}$$

and obtain

$$\begin{aligned} \max \quad & \delta \\ \phi_i(x) - b_i \delta \quad & = 0 \\ & 0 \leq x_{ij} \leq c_{ij} \end{aligned}$$

§4.3.2 Min cut

Definition 4.6 (Cuts). A *cut* of a graph G is a partition of V into S and $V \setminus S$. The *capacity* or *value* of the cut is

$$\sum_{\{(i,j) \in E: i \in S, j \in V \setminus S\}} c_{ij}$$

Theorem 4.7

Let x be a feasible flow with flow value δ . Then, for any cut $(S, V \setminus S)$ such that $1 \in S, n \in V \setminus S$, we have

$$\delta \leq C(S)$$

Remark. It is immediate that the max flow must be less than or equal to the min cut.

Proof. For any sets $A, B \subseteq V$, write

$$f_x(A, B) = \sum_{\{(i,j) \in E: i \in A, j \in B\}} x_{ij}$$

which is defined regardless of whether A, B are disjoint. Now, summing up the flow constraints over $i \in S$, we have

$$\sum_{i \in S} \left(\sum_{\{j: (i,j) \in E\}} x_{ij} - \sum_{\{j: (j,i) \in E\}} x_{ji} \right) = \delta$$

which gives

$$\begin{aligned} \delta &= f_x(S, V) - f_x(V, S) \\ &= f_x(S, S) + f_x(S, V \setminus S) - f_x(S, S) - \underbrace{f_x(V \setminus S, S)}_{\geq 0} \\ &\leq f_x(S, V \setminus S) \\ &\leq C(S) \end{aligned}$$

■

Theorem 4.8

Let δ^* be the max value of the flow. Then

$$\delta^* = \min\{C(S) : S \subset V, 1 \in S, n \in V \setminus S\}$$

Proof. It suffices to produce a set S such that $c(s) = \delta^*$, by the previous theorem. We define an *augmenting path* to be a path from 1 to n , ignoring edge orientations, such that every edge traversed forward satisfies $x_{ij} < c_{ij}$ and every edge traversed backwards satisfies $x_{ij} > 0$.

It is clear that an optimal solution to the flow problem cannot have an augmenting path insofar as forward edges are concerned. For the backward edges, we can subtract off a bit of the backward flow and hence get more flow to the sink. Hence an optimal solution can have no augmenting path. Set

$$S = \{1\} \cup \{i \text{ s.t. there is an augmenting path } 1 \text{ to } i\}$$

which satisfies $1 \in S$, $n \notin S$. For this set, we have that, summing up the flow constraints as before,

$$S^* = f_x(S, V \setminus S) - f_x(V \setminus S, S)$$

Observe that, for all forward paths S to $V \setminus S$, it must be that $x_{ij} = c_{ij}$, as otherwise we could construct an augmenting path – likewise, every backward path must have $x_{ij} = 0$. Hence, $f_x(S, V \setminus S) = C(S)$, while $f_x(V \setminus S, S) = 0$, giving that $\delta^* = C(S)$ as required. ■

Remark. In the same way we used the dual problem as a certificate of validity for our proposed solutions to linear programs, we can construct a cut to act as a certificate of validity for proposed solutions to the max flow problem. Indeed, this is called the *Ford-Fulkerson algorithm*.

In fact, this algorithm is not a perfect application of simplex, and it is not guaranteed that it will ever converge to the solution.